

Trading Strategy Parameter Optimization — Full Project Summary

This document summarizes an extended project analyzing and tuning entry-filter parameters for a family of related trading strategies (`AtsPriceQuickReversal`, `AtsPriceBrkout`, `AtsFastReversal`, `AtsSlowReversal`). It covers the tools built, the methodology established along the way, every confirmed and rejected finding per strategy, and the deep-dive analysis at the end of the thread. Use this as a standalone reference — it does not assume you have the original conversation available.

1. The strategies

All four strategies share the same skeleton: a fast/slow Hull Moving Average (HMA) pair, a set of boolean "C" condition flags, and (in the newer two) two weighted composite scores combined through an OR-gate.

AtsPriceQuickReversal — the original strategy. Enters on a sharp, sudden reversal (a "speed flip").

AtsPriceBrkout — similar structure, enters on breakout continuation instead of reversal. Its `C5`/`C12` definitions differ from `QuickReversal`'s and turned out to be **constant** (always true) in its trade log — a strategy-specific finding, not a general pattern.

AtsFastReversal — requires a *strong, already-extended* reversal. `ATRsFromHma` matters a lot here (confirmed repeatedly, short side especially).

AtsSlowReversal — sibling to Fast, but catches a move that has *already started* turning (less extension needed). `ATRsFromHma` does **not** matter here — a designed difference, not a gap in analysis.

The entry mechanism (Fast/Slow, current formula)



```
PatternEntryScore = IFF(C5, 1, 0)           // Speed flip
                  + IFF(C6 Or C10, 1, 0)     // At least 1 CVD confirmation
                  + IFF(C14 Or C15, 3, 0)     // Bar formation pattern

CVDEntryScore    = IFF(C6, 2, 0)           // CVDSpeedPct
                  + IFF(C10, 4, 0)         // CVD accel last 2 bars
                  + IFF(C13, 1, 0)         // CVDDeltaPct confirms
                  + IFF(C11, 2, 0)         // ATRsFromHma bar expansion

C3 = PatternEntryScore >= MinPatternEntryScore (4)
C12 = CVDEntryScore    >= MinCVDEntryScore

Entry: If (C3 Or C12) And C7 And C8 Then ...
```

`C3` and `C12` are two independent evidence paths (pattern-based vs. CVD/volume-based) — a trade can qualify on either alone. This is why the toolchain has an **entry-path stratification** feature: pooling Pattern-triggered and CVD-triggered trades can hide effects specific to one path.

`C7 = PipSpeedTrendPct >= HMinPipSpeedTrendPct` and `C8 = PipSpeed >= PipSpeedLimit` (or `AbsValue(PipSpeed) >= PipSpeedLimit` for short) are separate, mandatory hard gates layered on top.

2. The toolchain built

Six files, all sharing the convention `ProfitHit = Profit/Loss > 0` (every exit type counts), long/short always analyzed separately, and every finding labeled with a confidence level rather than presented as uniformly trustworthy.

`optimizer_constants.py` — shared foundation

`Column(str, Enum)` — every CSV column name, defined once

`TradeDataLoader` — CSV loading, `ProfitHit` derivation, optional chronological sort, direction splitting

`ColumnSelector` — parametrized "which columns are tunable parameters" logic (handles real differences between scripts)

`EntryPathAnalyzer` — the `pattern_only`/`cvd_only`/`both` stratification logic, shared across scripts instead of duplicated

`ats_param_optimizer.py` → `GridSearchOptimizer`

Grid sweep: Mann-Whitney significance test per parameter, single-parameter and 2-parameter combo expectancy sweeps, boolean flag tests, `--min-n` overfitting guard (default 30). `--compare-filter` forward-tests one specific rule against a dataset directly — **note: this only prints to console, it is not written into the `--output` JSON**, a real limitation discovered mid-project.

`ats_optuna_optimizer.py` → `BayesianThresholdOptimizer`

Bayesian (TPE) joint threshold search across all parameters at once, with a genuine chronological train/test split (`--test-fraction`) and optional in-training CV (`--cv-folds`). This has repeatedly caught real overfitting — filters that look great in training and then collapse (even flip sign) on held-out data.

`ats_feature_importance.py` → `FeatureImportanceAnalyzer`

Random Forest / Gradient Boosting + permutation importance + optional SHAP. Reports cross-validated AUC first; explicitly flags $AUC \approx 0.5$ (no signal) and $AUC < 0.5$ (noise, not an "inverse signal") rather than presenting a ranking as meaningful when it isn't. Skips modeling below 60 trades per direction.

`ats_entryscore_weight_optimizer.py` → `ParameterOptimizer`

Optimizes integer weights per EntryScore component + a score threshold (different search problem than the other two — shared weight per component, not independent thresholds). Supports `--components-long`/`--components-short` (needed for direction-flipped signs like `CVDDelta`) and an `"or"` comparison type (`col1|col2:or`) for terms like `C6 Or C10`. `--min-threshold-frac` guards against a **degenerate-threshold pitfall**: several components can share a high weight so any one alone still clears a raised threshold — functionally a no-op filter wearing a complicated disguise. Always check the "optimized" filter's `train_n` against the full training window size to catch this.

`ats_performance_report.py` → `PerformanceReportGenerator`

TradeStation-style performance report (console + Excel). Sheets: Trade Summary, Daily Performance, Performance By Symbol, and — added later — **Performance By Interval** plus one full Trade Summary sheet **per tick interval**, letting you check whether a strategy holds up consistently across different bar intervals rather than only looking at pooled numbers.

3. Hard-won methodology rules

Apply these before trusting any new result — each was learned the hard way during this project.

Always verify a JSON/report against the actual current CSV. Several times a report turned out to be

generated from an older, smaller version of the trade log even though it was just uploaded. Check `csv_path` and baseline `n` before trusting a report.

Train-window numbers are not evidence. Only test-window (held-out) results, at adequate sample size, should inform a live parameter change. Train-good/test-bad collapse is the single most common failure mode across every tool in this project.

A significant p-value with identical hit/loss medians is a red flag. Seen repeatedly — usually means a lumpy/discrete distribution skewing the rank test, not real separation. Always eyeball the medians directly.

"Beats baseline" is not the same as "profitable." A filter can beat a losing baseline while still losing money itself.

Raw, price-scale-dependent quantities are confounded by instrument price level. Prefer ATR-normalized counterparts (`DeltaATRs` over `DeltaPips`) as decision gates — **with one important exception found late in this project:** on the most recent FastReversal long dataset, `DeltaATRs` did NOT reproduce `DeltaPips`'s edge at any threshold tested (best `DeltaATRs` cut reached ~\$3/trade vs. `DeltaPips>=100`'s \$10.51/trade). This suggests the `DeltaPips` edge may be partly a symbol/price-level effect rather than a clean, transferable volatility-adjusted signal — worth extra caution before generalizing it.

Single-bar or short-window raw ATR values should not be standalone decision gates — normalization only, in general. **Partial exception found late in this project:** for FastReversal long, both `AvgATR` and `BarATR` independently and monotonically predict better outcomes as they rise, and are correlated (0.637) — likely reflecting one underlying "is this a genuinely volatile regime" signal rather than two independent ones. Treat this as a case-by-case exception, not a reversal of the general rule.

A component with 0% or 100% firing rate carries zero information — confirmed repeatedly (`BrkOut` C5/C12 at 100%, C9 at 0%; FastReversal's `C7`/`C8` frequently found at 100% across multiple datasets). Always check firing rate before including a flag in any search.

One boolean flag can be a strict logical subset of another. Found late in this project: `C14=1` implies `C15=1` in every single trade (`C14`'s firing count exactly equals the count where both fire). This makes `(C14 Or C15)` mathematically identical to `C15` alone — `C14` currently contributes nothing to the OR. Worth checking directly in source code whether this is by design or a bug.

A "confirmatory" condition can turn out to correlate with worse outcomes. Also found late: trades where `C15=0` (the bar-pattern condition does *not* fire) outperformed `C15=1` trades in **both** directions (long: \$3.71 vs -\$0.03; short: \$13.40 vs \$2.92). The intended positive-scoring condition may currently be counterproductive — flagged as a priority item to investigate in the strategy's source logic.

A fixed scalar "unit conversion" between a raw and normalized metric rarely holds exactly. Tested directly: converting `PipSpeedLimit(0.90)` to a proposed `PipSpeedNorm` threshold of `9` (a "×10" assumption) only reproduced the original filter's classification 81% of the time, and actively hurt long-side performance at every threshold tested. Always verify a proposed normalized-equivalent threshold empirically rather than assuming a clean multiplier.

The same "C" flag number means different things across strategy versions. `C5`, `C9`, `C12`, `C13` in particular have meant different conditions in `QuickReversal`/`BrkOut` vs. the current `Fast`/`Slow` formulas. Always check the specific strategy's own current source.

CVD-family conditions can have opposite sign conventions for long vs. short, even under "the same" formula — confirmed for `C10` (`CVDSPeetPct`): short's `C10=1` cleanly matches negative values (consistent with the stated formula), but long's `C10=1` showed exclusively **positive** values — the mirror image, not a literal same-sign application.

Tick-interval choice can matter more than any single parameter. One dataset showed 20-tick clearly outperforming 10/30/40/50-tick bars for the same strategy and symbols. The mechanism proposed (ATR volatility triggering premature stops at smaller intervals) was tested directly and **not confirmed** — stop-loss frequency was virtually identical across intervals (~98% both), and *median* stop-loss size was nearly identical too (the initial *mean*-based gap turned out to be driven by outlier/tail skew, not a systematic difference). The interval effect is real in the data checked so far but its cause remains unresolved.

4. Per-strategy findings (confirmed vs. rejected)

AtsPriceQuickReversal

Confirmed, replicated across independent batches: Long — `FullDeltaATRs >= 9 AND FullAngle >= 26`.

Confirmed mechanism: `PipSpeedTrendPct` needs a **ceiling** (~60-65 long, ~50-60 short), not a higher floor — it measures how uniformly fast the *preceding* trend leg was; a high value means a still-accelerating move (risky to fade), a lower value means genuine exhaustion (the strategy's premise).

Confirmed direction, recalibrated: `HMAgapCV` — keep `<=` (verified correct against an earlier proposed `>=` "fix" that was tested and rejected), but the real useful range is ~1.0-1.6, not the original 0.40.

Not reliably replicated: short-side `ATRsFromHma` finding — treat with caution.

EntryScore reweighting: consistently shows no benefit on the long side across growing datasets — equal weights already capture nearly all available value there.

AtsPriceBrkout

Confirmed: Long — significant on `ATRsFromHma` and `FullDeltaATRs` (both higher=better); best combo `FullDeltaATRs>=10.04 AND RevATRsPerSec<=0.63`.

Confirmed: Short — significant cluster around `PipSpeed`/`PipSpeedAce1`/`PipSpeedAce1Norm`; best combo `PipSpeed>=-0.68 AND ATRsFromHma>=0.49`.

`HMAgapCV<=0.40` is **dead** (0% firing both directions — BrkOut's typical values sit around 1.8-1.9). Short benefits from `<=1.0`; **long shows no benefit from any HMAgapCV filter** — don't add it there.

`C5`/`C12` are **structurally constant (100%)** — exclude from any search; this differs from QuickReversal, where `C5` is a real signal.

AtsFastReversal

Confirmed, held-out validated (strongest single result in the whole project): Short — `ATRsFromHma >= 1.5` (p=0.002, monotonic threshold sweep, and — critically — confirmed via chronological train/test split: training \$1.59/trade, test \$12.24/trade on n=24, beating the *same period's own baseline* by a wide margin). Layering `DeltaATRs<=7.7` or `PipSpeedTrendPct>=68.5` on top adds further held-out-confirmed lift.

Confirmed on a separate, larger dataset: Long — `DeltaPips >= 100` (clean monotonic sweep) and `AvgATR >= 0.12`, combo reaching \$11.32/trade (n=75) vs. \$1.05 baseline. See methodology section for the caveats found on both (DeltaATRs doesn't replicate DeltaPips's edge; AvgATR's mechanism is plausible but overlaps heavily with BarATR).

Correction to an earlier general assumption: `ATRsFromHma` does **not** currently help FastReversal long (p=0.52, no signal, and the threshold sweep trends the wrong way) — it's a short-only lever right now, not the universal one it was treated as earlier.

PipSpeedTrendPct formula changed mid-project (lookback window extended from 5 to 7 bars) — a genuinely different indicator under the same name; findings from before/after this change aren't directly comparable.

HMAGapCV **sentinel-rate regression discovered:** went from ~0% sentinel hits (well-calibrated) to 68-73% on a later, larger dataset — **HMinGapMean** likely needs recalibrating for Fast now too (previously a Slow-only problem).

C7/C8 frequently found at or near 100% firing across multiple datasets in this project — recurring near-no-op gates.

Interval analysis: 20-tick outperforms 10/30/40/50-tick on the datasets checked; mechanism unconfirmed (see methodology section).

Deep-dive findings from the final analysis session:

C3 (Pattern path) is the better path for long (+\$2.00 expectancy); **C12** (CVD path) is clearly the better path for short (+\$9.37 vs. Pattern's +\$4.03) — but **C12** is actively net-negative for long (-\$2.36) despite an OK hit rate, meaning its losses run larger when wrong.

Tightening **CVDSpeedPctLim** does not help either direction — every tested threshold underperforms the current (~0) baseline.

C5 (speed flip) currently helps long (+\$5.70 vs -\$0.45 off) but **hurts** short (+\$2.21 vs +\$6.23 off) — an asymmetry worth fixing independent of any unit/normalization question.

C14 is a strict logical subset of **C15** (see methodology section) — **(C14 Or C15)** is currently equivalent to **C15** alone.

C15=1 trades underperform **C15=0** trades in both directions — the bar-pattern condition may currently be counterproductive as scored.

AtsSlowReversal

Confirmed: Long — **CVDAce1Pct >= 40** (p=0.0093, "hard gate"-level significance, clean single-param sweep, and anchors the best combo **CVDAce1Pct>=30 AND PipSpeedAce1Norm>=-0.97**).

Confirmed mechanism (opposite of Fast, as designed): **PipSpeedTrendPct** needs a **ceiling** (~71) for Slow, vs. Fast's floor — matches the strategies' different premises exactly. **TrendBarCount<=14** and

HMAGapCV<=1.9-3.5 also independently confirmed for long via significance + feature-importance agreement.

Real but not yet actionable: Short — **CVDAce1Pct** and **RevATRsPerSec** both reach real significance, but with only ~32 total short trades and the min-n=30 guard, neither can produce a usable threshold yet. This is a "wait for more data" situation, not a "nothing works" one.

HMinGapMean=0.3 **badly miscalibrated for Slow specifically** — 95% of Slow's trades hit the **99** sentinel (**HMAGapMean** structurally smaller for Slow, since it catches earlier-stage turns with less-developed HMA gaps). Once **HMAGapMean** was logged directly (a mid-project CSV schema fix), the real distribution showed median ~0.10, meaning **HMinGapMean** should be closer to ~0.05, not 0.3.

Optuna and entry-score weight searches consistently failed to add value on this strategy specifically — one Optuna result was even *worse than baseline on its own training data*, a genuinely bad search outcome, not just overfitting.

5. Data-quality issues found and resolved

PatternEntryScore/C3 formula mismatches, found on two separate occasions: reconstructing the score from raw components didn't match the logged value (55-65% match rate at best on Fast; the Slow dataset showed logged scores of 6-7, mathematically impossible under the stated 5-point- max formula) — clear evidence a backtest was running an older formula version than described.

CSV header/data misalignment, twice: once a missing `HMAGapMean` header (verified as merely un-logged, not a real misalignment, since header/data field counts matched); once a genuine off-by-one (header had 61 fields, data rows had 60) caused by an empty trailing `ind_computertime` field being dropped rather than left blank — verified as benign (pandas handles a missing *trailing* field gracefully) rather than a real corruption. **`ind_C1` misread as non-binary** — turned out to be a display artifact from whatever tool was used to spot-check the raw file; confirmed always `1` when read properly through pandas.

6. Open items

Confirm whether `C14` is intentionally a logical subset of `C15`, or a bug — the single highest-priority open question from the final session.

Investigate why `C15=1` correlates with worse outcomes than `C15=0` in both directions for FastReversal — the bar-pattern condition may need redefinition, not just reweighting.

Fix the `C5`/short asymmetry for FastReversal (currently helps long, hurts short).

Recalibrate `HMinGapMean` for FastReversal given the sentinel-rate regression.

Log lagged `PipSpeed[1]`/`PipSpeed[2]` (or their normalized versions) to properly calibrate the proposed `PipSpeedFlipLimit` for `C5` — currently unverifiable from the data available.

Resolve the interval-effect mechanism (20-tick outperforming others) — ruled out simple ATR/stop-frequency explanations; cause still open.

Gather more AtsSlowReversal short trades before attempting any parameter change there — real signals exist but aren't yet actionable at current sample size.

Consider differentiating `C11`/`ATRsFromHma`'s weight between Fast and Slow's `CVDEntryScore` explicitly, since it matters by very different amounts between the two strategies.